

Jarvis Libras IA

Relatorio de controle do projeto

Visao computacional + IA temporal + Libras + voz + interface
Python | OpenCV | MediaPipe | TensorFlow/Keras | LSTM | CustomTkinter
Projeto em andamento - documentacao de comandos e proximos passos

Gerado em 13/05/2026

1. Resumo do projeto

Este documento resume o que foi construído no projeto Jarvis Libras IA, quais comandos foram usados no terminal, quais bibliotecas precisam estar instaladas, quais arquivos fazem parte do sistema e quais são os próximos passos recomendados. A ideia é servir como um controle do projeto para evitar perda de contexto e facilitar reinstalação, manutenção e evolução.

O sistema reconhece sinais em Libras pela câmera, extrai landmarks das mãos com MediaPipe, usa um modelo LSTM treinado com TensorFlow/Keras para identificar o sinal e traduz o resultado para texto e voz. Também existe um módulo separado de texto para Libras, usando interface com caixa de texto e vídeos/animações como avatar visual.

Nível atual

- Projeto em andamento, mas já funcional em protótipo.
- Reconhecimento em tempo real funcionando com modelo próprio treinado.
- Sistema já possui voz neural, interface dark e painel lateral para não cobrir a câmera.
- O foco atual é melhorar precisão, reduzir falsos positivos e ampliar o vocabulário com dataset pronto.

2. Tecnologias utilizadas

Tecnologia	Uso no projeto
Python	Linguagem principal do projeto.
OpenCV	Captura de câmera, exibição da interface e processamento dos frames.
MediaPipe	Deteção das mãos e extração dos 21 landmarks por mão.
NumPy	Organização dos vetores de landmarks e arrays do dataset.
TensorFlow/Keras	Criação, treino e carregamento do modelo de IA.
LSTM	Rede neural temporal para aprender movimentos/sequências.
scikit-learn	Separação treino/teste com <code>train_test_split</code> .
edge-tts + pygame	Voz neural mais natural em português do Brasil.
CustomTkinter	Interface moderna para texto -> Libras.
Git/GitHub	Versionamento e publicação do projeto.

3. Estrutura recomendada de pastas

```
jarvis-libras/  
├── assets/ # videos/imagens/avatar para texto -> Libras  
├── dataset/ # dataset convertido em .npy para treino  
├── dataset_videos/ # videos brutos do dataset pronto  
├── models/ # modelo treinado libras_model.h5  
├── Logs/ # logs do TensorBoard  
├── src/  
│   ├── collect_dataset.py # coleta dataset pela webcam  
│   ├── video_to_dataset.py # converte videos para landmarks .npy  
│   ├── train_model.py # treina o modelo LSTM  
│   ├── predict_realtime.py # reconhecimento em tempo real  
│   └── text_to_libras.py # texto digitado -> videos/avatar Libras
```

```
| └─ voice.py # voz neural/TTS  
└─ requirements.txt  
└─ README.md  
└─ .gitignore
```

4. Comandos principais usados no terminal

Criar e ativar ambiente virtual

```
py -3.11 -m venv venv  
venv\Scripts\activate  
python --version
```

Instalar bibliotecas principais

```
pip install opencv-python mediapipe numpy pandas  
pip install tensorflow scikit-learn  
pip install customtkinter  
pip install edge-tts pygame  
pip install pyttsx3
```

Corrigir conflitos comuns

```
pip uninstall protobuf -y  
pip install protobuf==5.29.5  
  
# Se TensorFlow continuar com erro:  
pip uninstall tensorflow protobuf -y  
pip install tensorflow==2.20.0 protobuf==5.29.5
```

Gerar requirements.txt

```
pip freeze > requirements.txt
```

Rodar os scripts principais

```
python src/video_to_dataset.py  
python src/collect_dataset.py  
python src/train_model.py  
python src/predict_realtime.py  
python src/text_to_libras.py
```

Git e GitHub

```
git init  
git add .  
git commit -m "Initial commit - Jarvis Libras AI"  
git branch -M main  
git remote add origin https://github.com/Anaiorino/jarvis-libras-ai.git  
git push -u origin main
```

Limpar Git quando o venv entrou errado no commit

```
rmdir /s /q .git  
rmdir /s /q .gitignore  
# depois recriar .gitignore como ARQUIVO, não pasta
```

Apagar dataset/modelo para reprocessar

```
rmdir /s /q dataset  
mkdir dataset
```

```
del models\libras_model.h5
```

5. Arquivos principais e funcao de cada um

Arquivo	Funcao
video_to_dataset.py	Converte videos de dataset_videos em sequencias .npy dentro de dataset/.
collect_dataset.py	Coleta sequencias pela webcam, util para sinais proprios ou classe neutro.
train_model.py	Carrega dataset, monta X/y, treina modelo LSTM e salva em models/libras_model.h5.
predict_realtime.py	Abre camera, extrai landmarks, faz predicacao com o modelo e mostra resultado no painel lateral.
voice.py	Faz o Text-to-Speech usando voz neural/edge-tts e correcoes de pronuncia.
text_to_libras.py	Interface para pessoa digitar texto e abrir videos/avatar em Libras.
requirements.txt	Lista bibliotecas instaladas para recriar o ambiente.
.gitignore	Impede subir venv, dataset, models e arquivos pesados para o GitHub.

6. Fluxo de treino atual

1. Organizar videos em dataset_videos/, com uma pasta por palavra.
2. Rodar video_to_dataset.py para transformar videos em landmarks .npy.
3. Conferir se dataset/// possui arquivos 0.npy ate 29.npy.
4. Atualizar o array actions em train_model.py e predict_realtime.py com a mesma ordem.
5. Rodar train_model.py para treinar e salvar models/libras_model.h5.
6. Rodar predict_realtime.py para testar na camera em tempo real.

7. Actions atuais do dataset pronto

```
actions = np.array([
    'neutro',
    'acontecer',
    'aluno',
    'amarelo',
    'america',
    'aproveitar',
    'bala',
    'banco',
    'banheiro',
    'maca',
    'vacina',
    'ruim',
    'conhecer',
    'espelho',
    'esquina',
    'medo',
    'sapo',
    'filho',
    'vontade',
    'barulho',
```

```
'cinco'  
])
```

Importante: a ordem das actions precisa ser identica no train_model.py e no predict_realtime.py. Se a ordem mudar, o modelo pode prever uma classe e o programa mostrar/falar outra.

8. Problemas que ja resolvemos

- MediaPipe sem mp.solutions por versao/Python - resolvido usando ambiente compativel.
- Camera presa por processo anterior - fechar janela/usar ESC ou CTRL+C.
- Git tentando subir venv e arquivos grandes - corrigido com .gitignore e reset do Git.
- TensorFlow/protobuf com conflito - corrigido instalando protobuf compativel.
- Voz robótica/travando - trocada para edge-tts/voz neural.
- Informacoes cobrindo a camera - criada interface com painel lateral.
- Predicao cedo demais no meio do movimento - adicionada confirmacao temporal.
- Falsos positivos sem sinal - adicionada classe neutro.

9. Configuracoes importantes no predict

```
SEQUENCE_LENGTH = 30 # quantidade de frames por sequencia  
PREDICT_EVERY = 5 # faz predicao a cada 5 frames para reduzir travamento  
CONFIDENCE_THRESHOLD = 0.90 # confianca minima para aceitar uma classe  
CONFIRMATION_FRAMES = 5 # precisa repetir a mesma palavra antes de falar  
PREDICTION_DELAY = 3 # intervalo minimo para falar novamente
```

10. Proximas implementacoes possiveis

- Tela inicial com botoes: Reconhecer Libras, Texto para Libras, Treinar IA, Configuracoes.
- Sistema de frases: juntar palavras reconhecidas e formar uma frase final.
- Botao para ativar/desativar voz.
- Historico das palavras reconhecidas.
- Modo calibracao de camera/iluminacao.
- Dashboard de acuracia por sinal depois do treino.
- Matriz de confusao para descobrir quais sinais estao parecidos.
- Exportar relatorio automatico do treino.
- Avatar 3D com Blender/Unity ou videos padronizados para texto -> Libras.
- Reconhecimento facial/expressao para sinais que dependem de expressao.
- Integracao com o Jarvis multimodal.
- Transformar em aplicativo desktop com PyInstaller ou interface completa em PyQt/CustomTkinter.
- Adicionar datasets maiores e mais pessoas para generalizar melhor.

11. Observacoes finais

O projeto esta em uma fase de prototipo avancado: ja tem pipeline completo de captura, dataset, treinamento, predicao, voz e interface. A principal limitacao atual e a precisao do reconhecimento, que depende diretamente da qualidade do dataset, equilibrio entre classes, quantidade de exemplos e diferenca visual entre os sinais.

Para continuar evoluindo sem se perder, sempre que adicionar novas palavras: padronize o nome da pasta, atualize actions nos dois arquivos, converta o dataset, treine novamente e teste com calma. Se um sinal confundir muito, colete mais exemplos desse sinal e do sinal com o qual ele esta confundindo.